Builder Center





Amazon ML Challenge 2026: Your Complete Prep Guide with Live Demo

Prepare for the Amazon ML Challenge 2026: free credits, live SageMaker demo, and more.



Jatin

AWS Employee Developer Advocate @ AWS | Dude who talks, build and break things on AWS

Follow

Published Sep 21, 2026 • Last modified Sep 25, 2026

98 126

TL;DR - 89,000+ students registered. $200 in free AWS credits for every participant. A 72-hour ML hackathon with prizes worth INR 2.25 lakh and PPIs at Amazon. I hosted a **prep session on September 21** to help you get started. Watch the recording on Twitch!

If you found this helpful, connect with me on LinkedIn! or instagram I regularly post about AWS, ML, GenAI and developer tools.

Note: *For all program related comms students should reach out to UnStop FAQ page / support email !!!*



> 📺 **WATCH RECORDING:** Watch the Recording on Twitch | **September 21, 5:00 - 6:00 PM IST**

What is the Amazon ML Challenge?

The **Amazon ML Challenge 2026** is a two-stage machine learning competition exclusively for **engineering students across India** (B.E./B.Tech/M.Tech/M.S./PhD, graduating 2027 or 2028). You form a team (2-4 members), get a real-world dataset from Amazon, and build an ML model to solve a business problem. Cross-college teams are allowed!

> IN **Who can participate?*** Students pursuing PhD/M.E./M.Tech/M.S./B.E./B.Tech at any engineering campus in **India*** Graduation year: **2027 or 2028*** Team size: **2-4 members** (cross-college teams allowed)* Must have a valid **AWS Builder Center Profile ID**

The Stakes

Reward	Who Gets It
INR 1,00,000 + certificates + goodies	🥇 Winners
INR 75,000 + certificates + goodies	🥈 First Runners-up

Reward	Who Gets It
INR 50,000 + certificates + goodies	🥉 Second Runners-up
Pre-Placement Interviews at Amazon	🎯 Top 50 Teams
$200 AWS Credits	💰 All Participants
Extra $100 Credits (at 48-hr mark)	⚡ Top 500 Teams
Certificates + Amazon SWAGs	🏅 Top 10 + Top 10 Women-only Teams

The Timeline

```
📋 Registration          → 7 - 22 Sep 2026
🔴 Prep Virtual Session → 21 Sep 2026, 5:00 - 6:00 PM IST ← RECORDING AVAILABLE
⚔️ 72-Hour Hackathon     → 25 - 27 Sep 2026
📊 Top 50 Results        → 2 Oct 2026
🏆 Grand Finale          → 7 Oct 2026 (Top 10 present to Amazon Scientists)
```

> 📺 **Watch Recording:** Watch the Recording on Twitch

What We Covered in the Live Session

I hosted a **Best Practices Virtual Session** live on Twitch to make sure you hit the ground running when the 72-hour clock starts. We covered **cloud fundamentals** (the core AWS services you need to know), **free resources** worth $579, and then I did a **live SageMaker demo** - training and deploying a real ML model, start to finish. Here's the breakdown:

1) AWS Builder Center - Your Free Dev Playground

Before anything else, every participant needs an **AWS Builder Center Profile**. It's mandatory for registration.

But Builder Center isn't just a checkbox - it's a genuinely useful platform:

Feature	Why You Care
🧪 Free Sandbox Environments	Real AWS accounts, no credit card. 8 hours per week.
📚 Hands-on Workshops	Level 100 to 400. Real services, not just videos.
🌐 Public Builder Profile	Portfolio URL for LinkedIn and resumes

Feature	Why You Care
👥 Student Builder Groups	Communities at universities in 60+ countries

> **How to set up:** Go to builder.aws.com → Join → Verify email → Pick an alias. Takes less than 5 minutes. No credit card.

2) Student Rewards - Up to $579 in Free Value

This is the one most students don't know about. AWS launched **Student Rewards** on August 20, 2026, and it's massive:

```
✅ Verify your student status
   └──→ 12 months Skill Builder Premium ($449 value)

🏅 Earn 7 badges on Builder Center
   └──→ $10 AWS Credits

🏅🏅 Earn 14 badges
   └──→ $20 AWS Credits

🏅🏅🏅 Earn 21 badges
   └──→ $100 AWS Certification Exam Voucher

💰 Total Value: $579
💳 Credit Card Required: NEVER
```

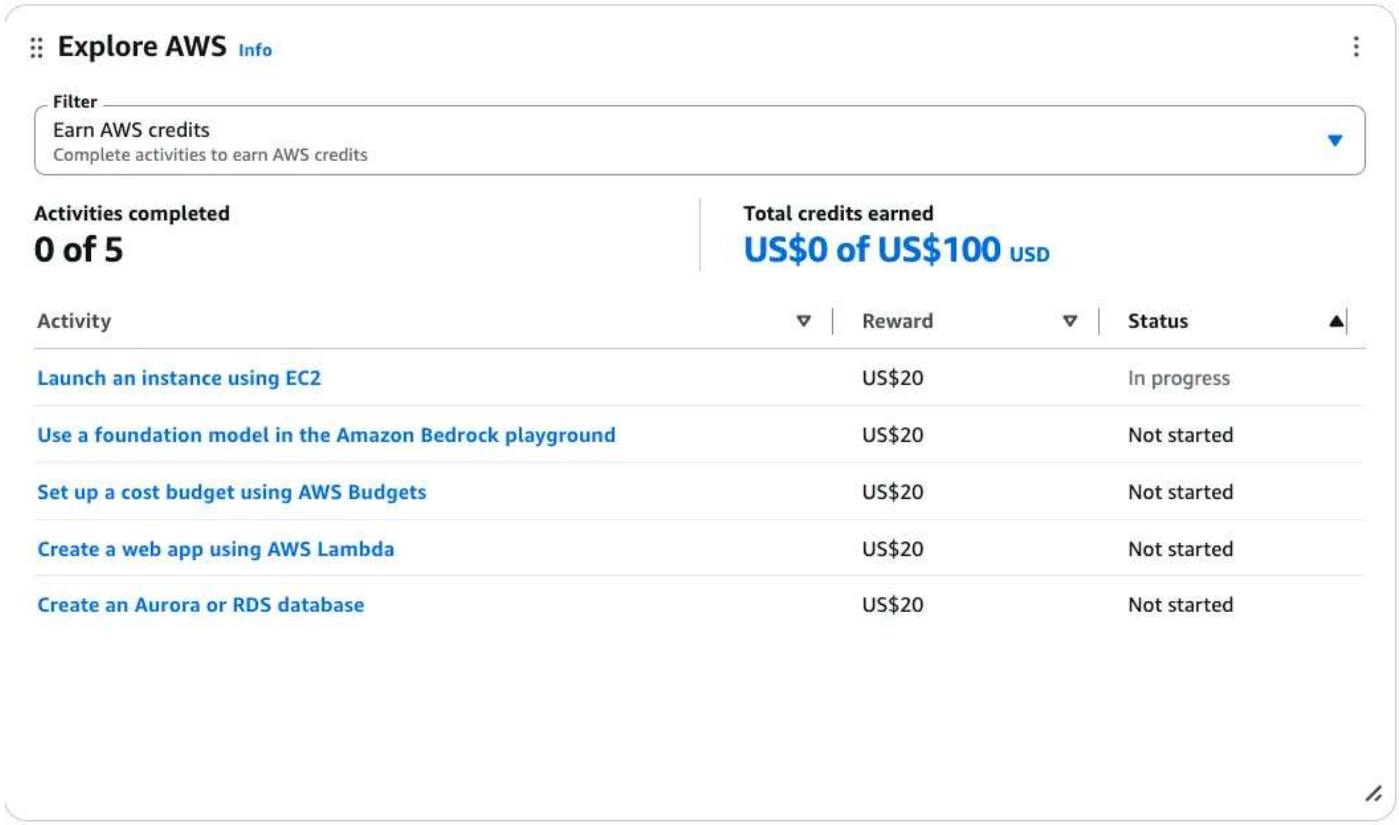
The badges come from simple daily activities - signing in, commenting, publishing articles, submitting feature requests. Most students can hit 7 badges in the first week.

> **Pro tip:** Start earning badges now. The $10 and $20 credits arrive early and can be used for SageMaker during the challenge.

3) AWS Free Tier - $0 to Get Started

Every participant receives **$200 in AWS credits** just by registering. But even without that, new AWS accounts come with a generous free tier:

Note: *Upon signing up you directly get $100 in AWS Credits and doing 5 activities to get you started how AWS works like creating EC2 Instance, or RDS database will get you more $100 so in total you get $200 in AWS Credits.*



Service	Free Tier
SageMaker Notebooks	250 hours on ml.t3.medium (2 months)
SageMaker Training	50 hours on ml.m5.xlarge (2 months)
SageMaker Inference	125 hours on ml.m5.xlarge (2 months)
S3 Storage	5 GB always free
Lambda	1M requests/month always free

Important caveats:

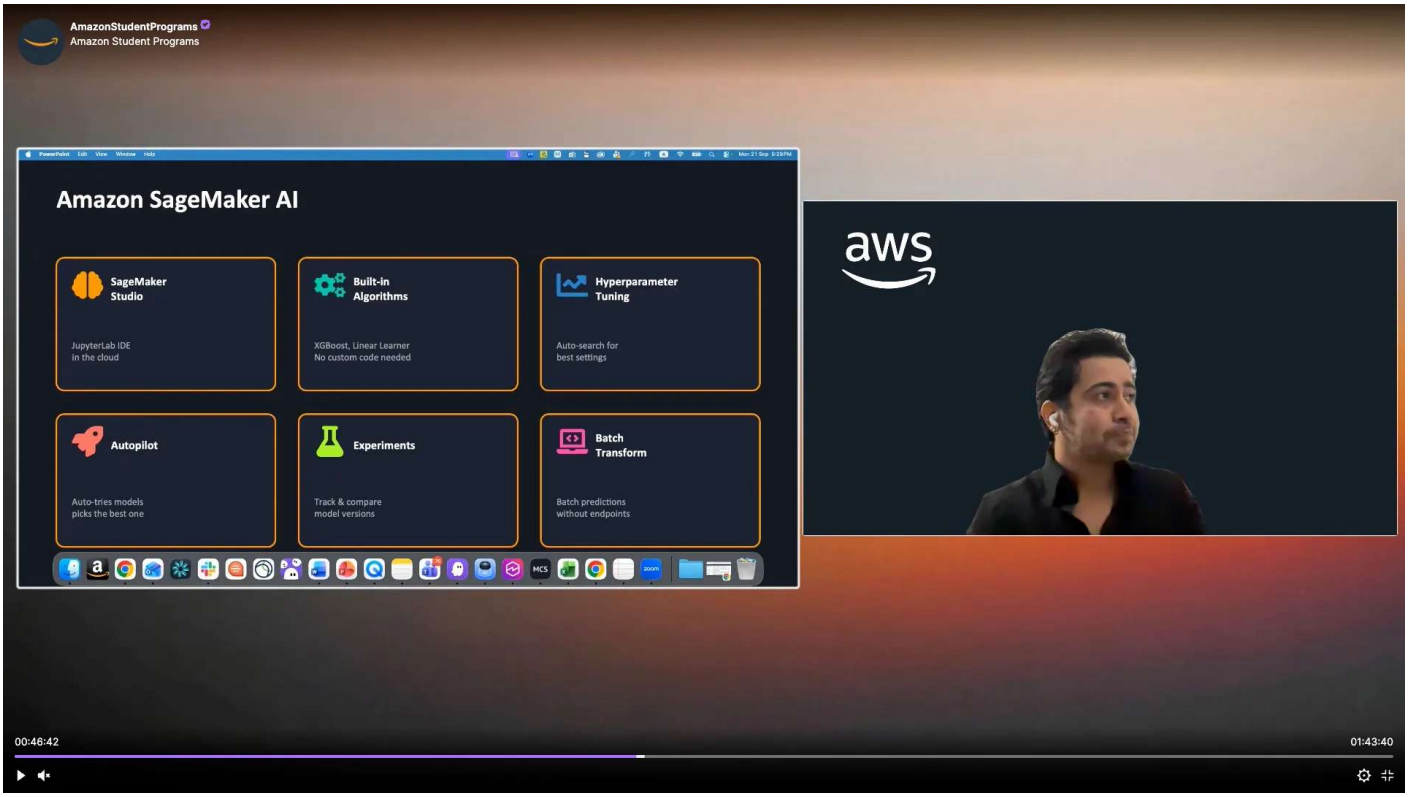
- Set **billing alerts** in the console immediately after creating your account
- Always **delete endpoints** when you're done (they charge ~$0.12/hour)
- Use **us-east-1** region for best compatibility

4) Cloud Fundamentals - The AWS Services You Need to Know

Before the demo, we're doing a quick walkthrough of the core AWS services you'll encounter:

Service	What It Does	Why You Need It
Amazon EC2	Virtual servers in the cloud	SageMaker training jobs run on EC2 instances behind the scenes
Amazon S3	Object storage (files, data, models)	Your data goes in, trained models come out. Everything flows through S3.
Amazon DynamoDB	NoSQL database	Store results, metadata, or feature stores at scale
IAM	Permissions and access control	The "role" that lets SageMaker access your data and spin up machines
Amazon VPC	Your private network inside AWS	All your SageMaker resources run inside a VPC. Think of it as your own private WiFi - keeps everything secure and isolated. Quick Setup creates one automatically

> You don't need to be an expert in all of these. But understanding what they do and how they connect will make the SageMaker demo (and the ML Challenge) click much faster.



> 📺 **Watch the Full Demo Recording! Everything below is what I'm demonstrating in the session. Follow along at Watch the Recording on Twitch - September 21, 5:00 - 6:00 PM IST**

Stream vs Blog: What's Different? [IMP, DON'T SKIP]

In the stream, we used SageMaker's managed infrastructure. In this blog, we do everything locally. Both work. Here's the difference:

Studio vs Notebook Instance - Studio is a full workspace with multiple apps but needs a Domain setup (which might take some time). A Notebook Instance is a simple Jupyter notebook - no Domain, no hassle. Same code runs on both.

Local Training vs Training Job - Local training runs inside your notebook using its own CPU. The data is already in memory, no S3 upload needed. A Training Job spins up a separate, more powerful machine that pulls data from S3. Use local when data is small. Use Training Jobs when you need big machines or GPUs.

Local Prediction vs Endpoint - Local prediction calls `model.predict()` instantly in your notebook. An Endpoint is a 24/7 hosted API for production apps - costs $0.12/hour even when idle.

For the ML Challenge: Notebook Instance + local training + local prediction. You submit a CSV, not a running API.

The Demo - Building an ML Model on SageMaker

The highlight of the session is a **30-minute hands-on demo** where we build, train, and predict with a machine learning model entirely on Amazon SageMaker - from raw data to live predictions. Here's every step, numbered, with the exact code.

Note: *In the live stream I have used SageMaker Studio to do the demo. Some people who have just created the AWS Free Tier Account as things are setting up behind the scenes might have to wait to use it but considering this is Time based challenge so feel free to use Notebook instance as i have done in the blog over here*

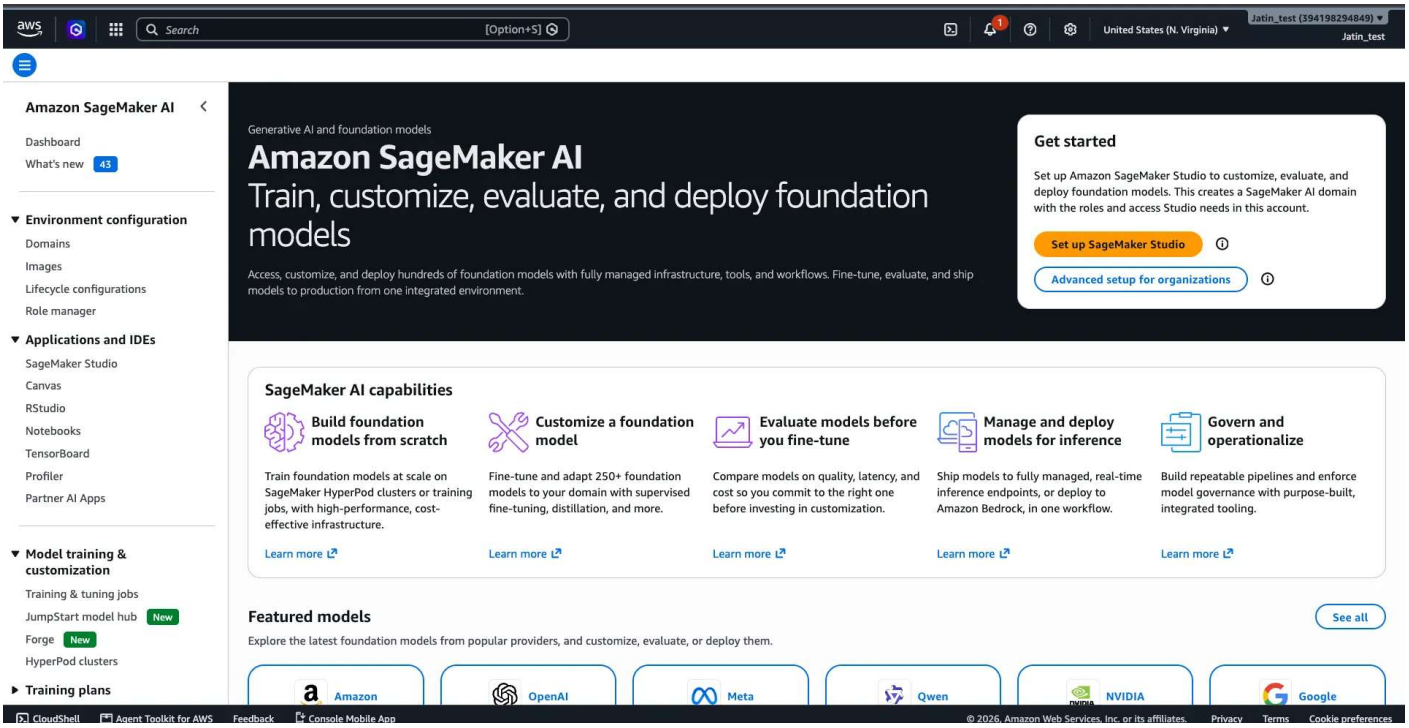
Other Folks can freely use their AWS account if they are using it from a long time, the demo from the stream will work perfectly.

In the stream i am training model on dedicated separate sagemaker training instance but in this code i am doing it inside the notebook instance only so you will find some differences in the code like upload to s3, deploying endpoint but both the demo works.

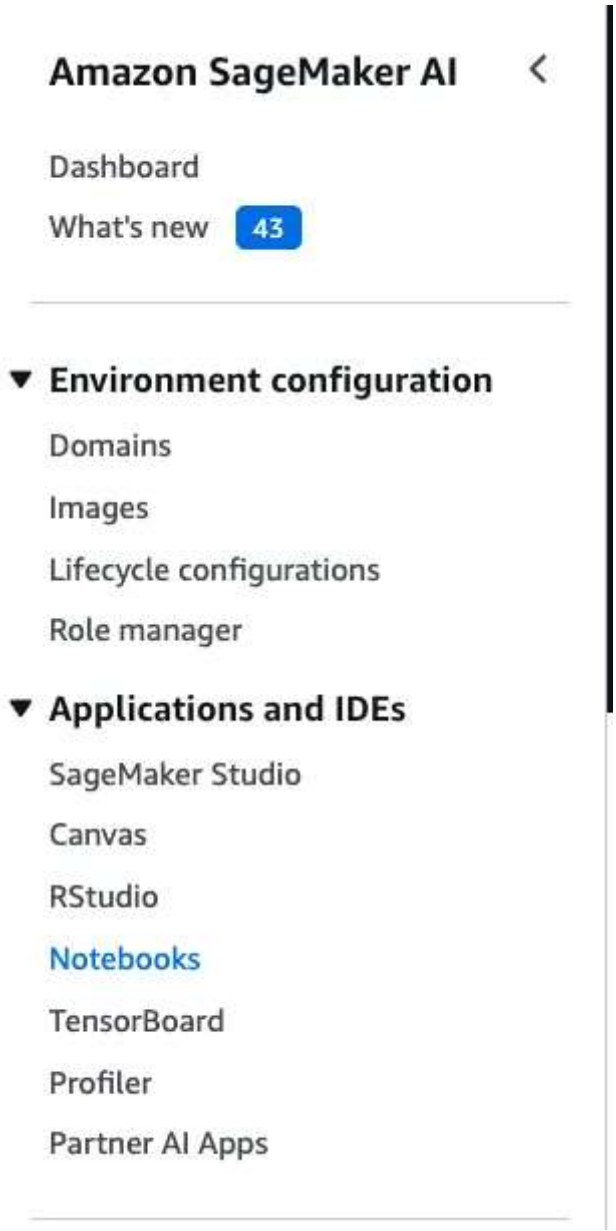
Step 0: Create a SageMaker Notebook Instance

Before writing any code, you need a notebook. We're using a **Notebook Instance** (not SageMaker Studio) since it works on all accounts without quota issues.

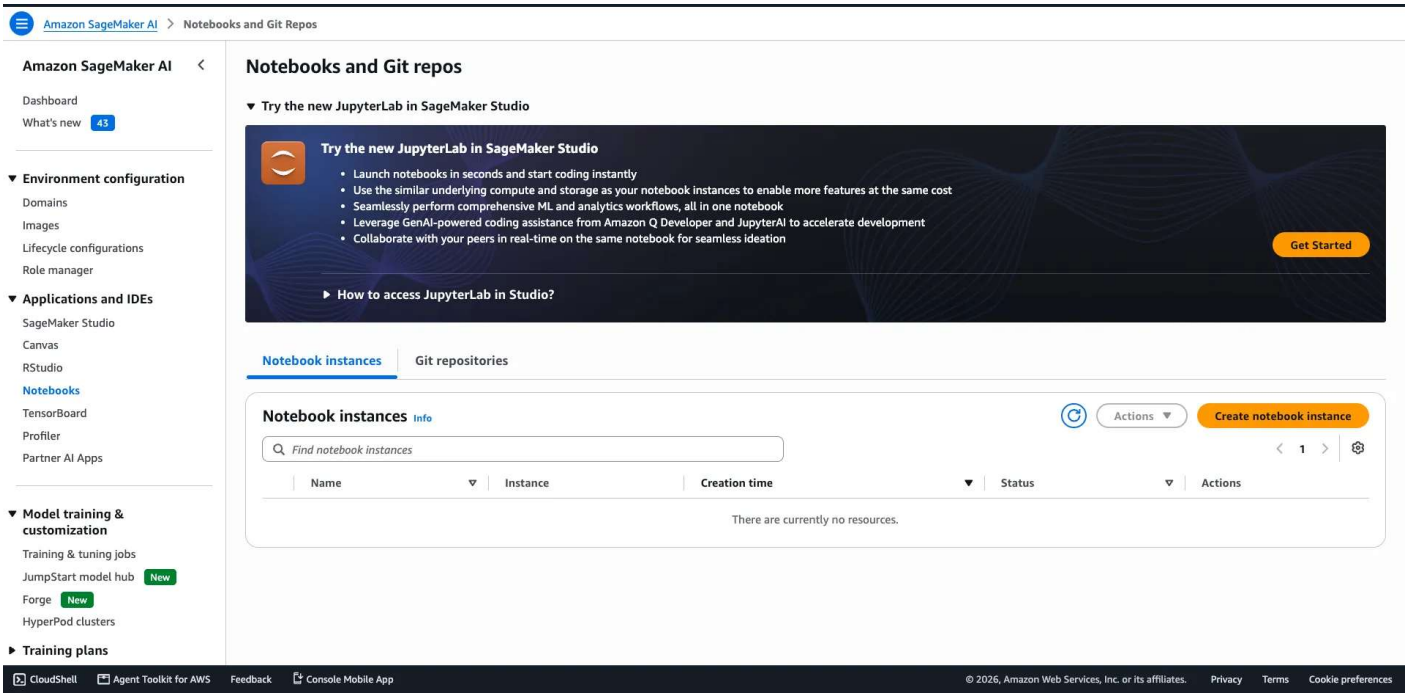
- Go to the **SageMaker AI** console



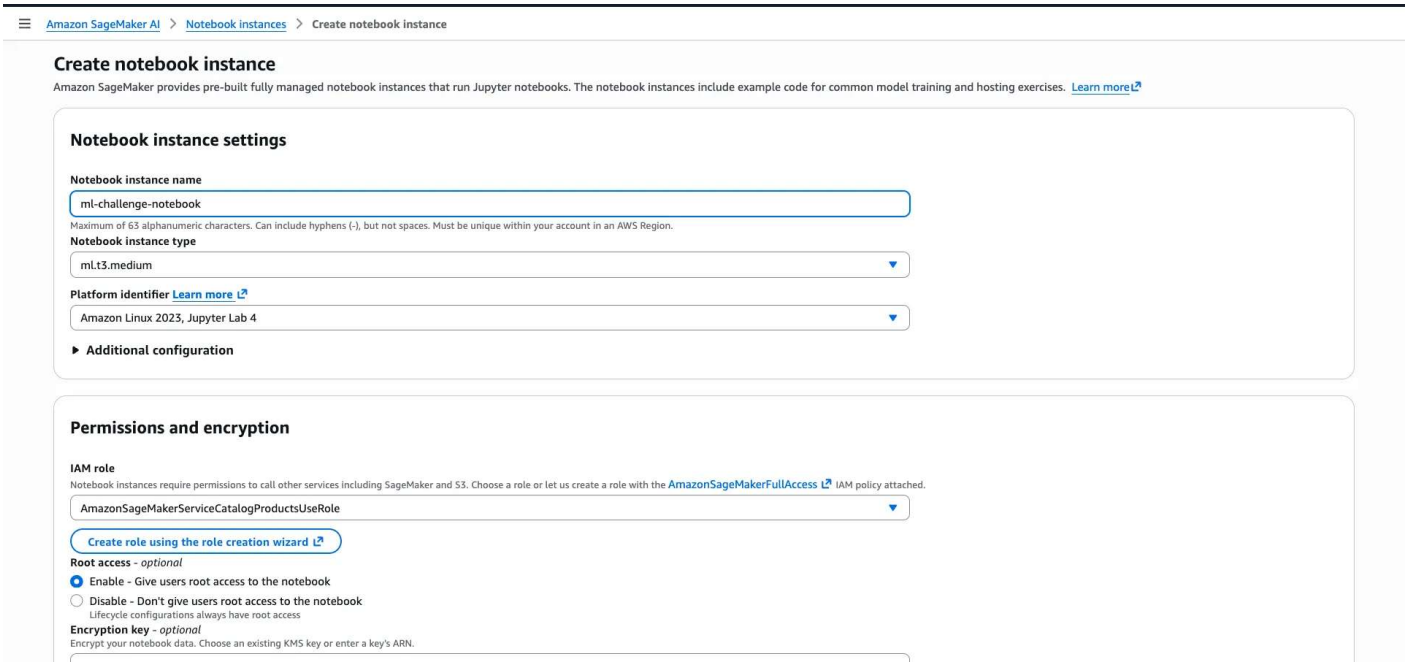
- In the left sidebar, click Under **Applications and IDEs -> Notebook -> Notebook instances**



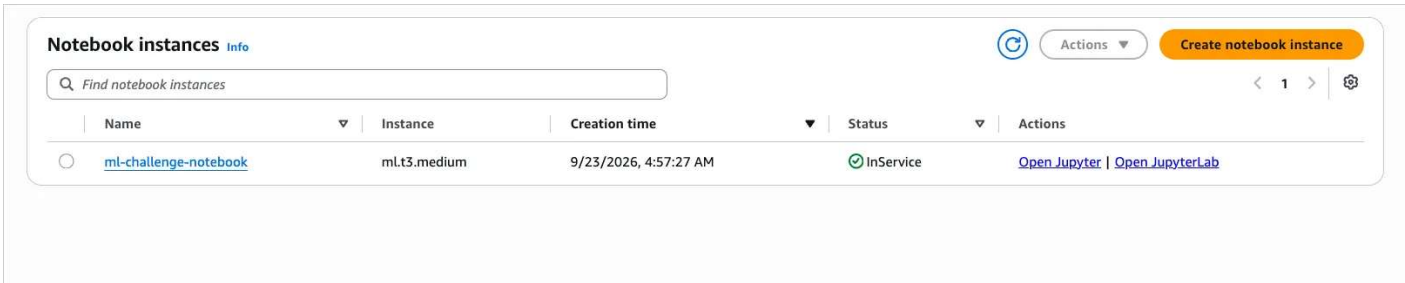
- Click **Create notebook instance**



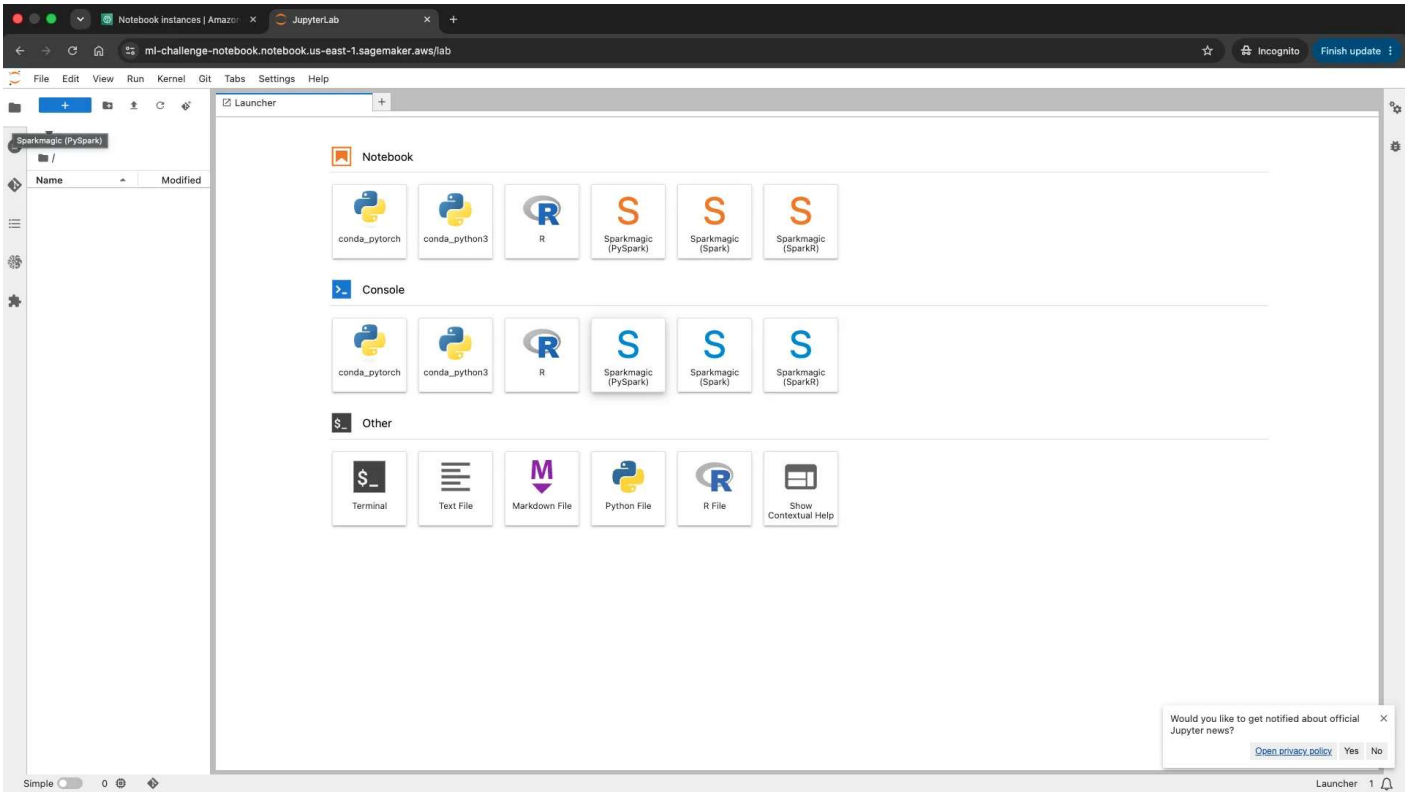
- Set the **Name** to any name of your choice, for this demo i chose `ml-challenge-notebook`



- Set **Instance type** to `ml.t3.medium` (covered by free tier - 250 hours)
- Under **IAM Role**, select **Create a new role** -> leave defaults -> click **Create role (Just chose default)**
- Click **Create notebook instance**
- Wait 2-3 minutes for status to change to **InService**



- Click **Open JupyterLab**
- Click the **"+"** button -> select **conda_python 3** notebook



Step 1: Install Libraries

```
!pip install xgboost scikit-learn -q
```

> This installs XGBoost (our ML algorithm) and scikit-learn (for metrics and data splitting). The `-q` flag keeps the output clean.

Note: You might think nothing is happening but observe the bottom line, it tells whether Kernel is busy or not.



Step 2: Set Up SageMaker Session

```
import sagemaker
import boto3
import pandas as pd
import numpy as np
import time

session = sagemaker.Session()
role = sagemaker.get_execution_role()
region = session.boto_region_name
bucket = session.default_bucket()

print(f"Region: {region}")
print(f"Role: {role}")
print(f"Bucket: {bucket}")
```



> **What each line does:*** `Session()` connects to SageMaker* `get_execution_role()` gets the permission pass (IAM role) that lets SageMaker access your data* `default_bucket()` is your auto-created S3 storage bucketThree lines and your entire infrastructure is ready.

Step 3: Load the Dataset

```
data = pd.read_csv(
    f"s3://sagemaker-example-files-prod-{region}/datasets/tabular/synthetic/churn.txt"
)
df = data.copy()

print(f"Dataset shape: {df.shape}")
df.head()
```

[8]:
```
data = pd.read_csv(
    f"s3://sagemaker-example-files-prod-{region}/datasets/tabular/synthetic/churn.txt"
)
df = data.copy()

print(f"Dataset shape: {df.shape}")
df.head()
```

```
/home/ec2-user/anaconda3/envs/python3/lib/python3.10/site-packages/fsspec/registry.py:305: UserWarning: Your installed version of s3fs is very old and known to cause
severe performance issues, see also https://github.com/dask/dask/issues/10276

To fix, you should specify a lower version bound on s3fs, or
update the current installation.

  warnings.warn(s3_msg)
Dataset shape: (5000, 21)
```

[8]:

	State	Account Length	Area Code	Phone	Int'l Plan	VMail Plan	VMail Message	Day Mins	Day Calls	Day Charge	...	Eve Calls	Eve Charge	Night Mins	Night Calls	Night Charge	Intl Mins	Intl Calls	Intl Charge	CustServ Calls	Churn?
0	PA	163	806	403-2562	no	yes	300	8.162204	3	7.579174	...	4	6.508639	4.065759	100	5.111624	4.928160	6	5.673203	3	True.
1	SC	15	836	158-8416	yes	no	0	10.018993	4	4.226289	...	0	9.972592	7.141040	200	6.436188	3.221748	6	2.559749	8	False.
2	MO	131	777	896-6253	no	yes	300	4.708490	3	4.768160	...	3	4.566715	5.363235	100	5.142451	7.139023	2	6.254157	4	False.
3	WY	75	878	817-5729	yes	yes	700	1.268734	3	2.567642	...	5	2.333624	3.773586	450	3.814413	2.245779	6	1.080692	6	False.
4	WY	146	878	450-4942	yes	no	0	2.696177	3	5.908916	...	3	3.670408	3.751673	250	2.796812	6.905545	4	7.134343	6	True.

5 rows × 21 columns

> Note: The dataset itself is a synthetic telecom customer dataset - about 5,000 customers with features like call minutes, charges, and customer service calls. **It's not real customer data - AWS generated it for learning purposes.** AWS hosts public datasets in S3 for tutorials. We're reading directly from their public bucket - no upload needed. For the ML Challenge, you'll upload your own dataset to YOUR S3 bucket and replace bucket url, of course you need to give your notebook IAM permission to do ;)

Step 4: Explore the Data

```
print("Target variable distribution:")
print(df["Churn?"].value_counts())
print(f"\nChurn rate: {df['Churn?'].value_counts(normalize=True)['True.']:.1%}")
```

```
print(f"Features: {df.shape[1]} columns, {df.shape[0]} rows")
print(f"Missing values: {df.isnull().sum().sum()}")
print(f"\nColumn types:\n{df.dtypes.value_counts()}")
```

```
[9]: print("Target variable distribution:")
     print(df["Churn?"].value_counts())
     print(f"\nChurn rate: {df['Churn?'].value_counts(normalize=True)['True.']:.1%}")

Target variable distribution:
Churn?
False.    2502
True.     2498
Name: count, dtype: int64

Churn rate: 50.0%

[10]: print(f"Features: {df.shape[1]} columns, {df.shape[0]} rows")
      print(f"Missing values: {df.isnull().sum().sum()}")
      print(f"\nColumn types:\n{df.dtypes.value_counts()}")

Features: 21 columns, 5000 rows
Missing values: 0

Column types:
int64      8
float64    8
object     5
Name: count, dtype: int64
```

Like explained in the stream as well

> Our target column is `Churn?` - True means the customer left, False means they stayed. This dataset has a roughly 50/50 split - about half churned and half stayed. This is an important step. Always check your target distribution before you do anything else. It tells you what kind of problem you're dealing with. Here it's balanced - roughly equal. But in the ML Challenge, your data might be heavily skewed to one side. That changes how you build and evaluate your model.21 columns, zero missing values. This is a clean dataset - the ML Challenge data won't be this clean! You'll need to handle missing values and noisy data there. But the exploratory analysis step is always the same - understand your data before you model it.

Step 5: Feature Engineering

```python
from sklearn.model_selection import train_test_split

# Drop non-predictive columns
df = df.drop("Phone", axis=1)

# Area Code is a code, not a number
df["Area Code"] = df["Area Code"].astype(object)

# Remove redundant charge columns (charges = minutes x rate)
df = df.drop(["Day Charge", "Eve Charge", "Night Charge", "Intl Charge"], axis=1)

# One-hot encode categorical features (Yes/No -> 1/0)
model_data = pd.get_dummies(df)

# Move target to first column (XGBoost convention)
model_data = pd.concat(
    [model_data["Churn?_True."],
     model_data.drop(["Churn?_False.", "Churn?_True."], axis=1)],
    axis=1
)
model_data = model_data.astype(float)

```

```
print(f"Processed: {model_data.shape}")
model_data.head()
```

[11]:
```
from sklearn.model_selection import train_test_split

# Drop non-predictive columns
df = df.drop("Phone", axis=1)

# Area Code is a code, not a number
df["Area Code"] = df["Area Code"].astype(object)

# Remove redundant charge columns (charges = minutes x rate)
df = df.drop(["Day Charge", "Eve Charge", "Night Charge", "Intl Charge"], axis=1)

# One-hot encode categorical features (Yes/No -> 1/0)
model_data = pd.get_dummies(df)

# Move target to first column (XGBoost convention)
model_data = pd.concat(
    [model_data["Churn?_True."],
     model_data.drop(["Churn?_False.", "Churn?_True."], axis=1)],
    axis=1
)
model_data = model_data.astype(float)

print(f"Processed: {model_data.shape}")
model_data.head()
```

```
Processed: (5000, 100)
```

[11]:

	Churn? _True.	Account Length	VMail Message	Day Mins	Day Calls	Eve Mins	Eve Calls	Night Mins	Night Calls	Intl Mins	...	Area Code_858	Area Code_866	Area Code_868	Area Code_876	Area Code_877	Area Code_878	Int'l Plan_no	Int'l Plan_yes	VMail Plan_no	VMail Plan_yes
0	1.0	163.0	300.0	8.162204	3.0	3.933035	4.0	4.065759	100.0	4.928160	...	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	1.0
1	0.0	15.0	0.0	10.018993	4.0	2.325005	0.0	7.141040	200.0	3.221748	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	1.0	0.0
2	0.0	131.0	300.0	4.708490	3.0	4.537466	3.0	5.363235	100.0	7.139023	...	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	1.0
3	0.0	75.0	700.0	1.268734	3.0	2.528748	5.0	3.773586	450.0	2.245779	...	0.0	0.0	0.0	0.0	0.0	1.0	0.0	1.0	0.0	1.0
4	1.0	146.0	0.0	2.696177	3.0	6.015337	3.0	3.751673	250.0	6.905545	...	0.0	0.0	0.0	0.0	0.0	1.0	0.0	1.0	1.0	0.0

5 rows × 100 columns

> **Key takeaway:** Feature engineering often matters more than your choice of algorithm. Spend 70% of your time here.

> Feature engineering - this is where data science happens. Let me walk through what each line does. First, I'm dropping Phone number. Why? It's a unique ID - every customer has a different phone number. It doesn't help predict anything. If your data has unique IDs, serial numbers, or row numbers - drop them. They add no value.I moved the target column to position 0, the very first column. Why? Because SageMaker's built-in XGBoost has a rule: give me a CSV file with no column headers, and put the thing you want to predict in the FIRST column. Everything else goes after it. This is the number one beginner mistake. People forget to remove headers, or they leave the target column somewhere in the middle. The model either crashes or trains on the wrong thing. So remember two rules: no headers, target first. Quick aside - someone might be wondering, how do I know these rules? The answer is **documentation** ;)

Step 6: Split the Data

```
# Split into train (67%), validation (22%), test (11%)
train_data, validation_data = train_test_split(model_data, test_size=0.33, random_stat
validation_data, test_data = train_test_split(validation_data, test_size=0.33, random_

# Separate test labels (we'll use these to check accuracy later)
test_target = test_data['Churn?_True.']
test_data_no_target = test_data.drop(['Churn?_True.'], axis=1)

# Separate features and labels for train/validation
```

```
train_features = train_data.iloc[:, 1:]
train_labels = train_data.iloc[:, 0]
val_features = validation_data.iloc[:, 1:]
val_labels = validation_data.iloc[:, 0]

print(f"Training:   {train_data.shape[0]} rows")
print(f"Validation: {validation_data.shape[0]} rows")
print(f"Test:       {test_data.shape[0]} rows")
```

[12]:
```
# Split into train (67%), validation (22%), test (11%)
train_data, validation_data = train_test_split(model_data, test_size=0.33, random_state=42)
validation_data, test_data = train_test_split(validation_data, test_size=0.33, random_state=42)

# Separate test labels (we'll use these to check accuracy later)
test_target = test_data['Churn?_True.']
test_data_no_target = test_data.drop(['Churn?_True.'], axis=1)

# Separate features and labels for train/validation
train_features = train_data.iloc[:, 1:]
train_labels = train_data.iloc[:, 0]
val_features = validation_data.iloc[:, 1:]
val_labels = validation_data.iloc[:, 0]

print(f"Training:   {train_data.shape[0]} rows")
print(f"Validation: {validation_data.shape[0]} rows")
print(f"Test:       {test_data.shape[0]} rows")
```

```
Training:   3350 rows
Validation: 1105 rows
Test:       545 rows
```

> **Why three sets?** Training data = textbook (model learns from it). Validation = practice tests (model checks progress). Test = final exam (never seen until the end). Golden rule: never touch test data until you're done experimenting.

Step 7: Train with XGBoost (Locally)

```

# Create XGBoost's special data format
dtrain = xgb.DMatrix(train_features, label=train_labels)
dval = xgb.DMatrix(val_features, label=val_labels)

# Hyperparameters
params = {
    "max_depth": 5,
    "eta": 0.2,
    "gamma": 4,
    "min_child_weight": 6,
    "subsample": 0.8,
    "objective": "binary:logistic",
    "eval_metric": "logloss"
}

print("Training locally... (no separate machine needed)")
model = xgb.train(
    params, dtrain, num_boost_round=100,
    evals=[(dtrain, "train"), (dval, "validation")],
    verbose_eval=10
)
print("\nTraining complete!")
```

[13]:
```
import xgboost as xgb

# Create XGBoost's special data format
dtrain = xgb.DMatrix(train_features, label=train_labels)
dval = xgb.DMatrix(val_features, label=val_labels)

# Hyperparameters
params = {
    "max_depth": 5,
    "eta": 0.2,
    "gamma": 4,
    "min_child_weight": 6,
    "subsample": 0.8,
    "objective": "binary:logistic",
    "eval_metric": "logloss"
}

print("Training locally... (no separate machine needed)")
model = xgb.train(
    params, dtrain, num_boost_round=100,
    evals=[(dtrain, "train"), (dval, "validation")],
    verbose_eval=10
)
print("\nTraining complete!")
```

```
Training locally... (no separate machine needed)
[0]     train-logloss:0.57772   validation-logloss:0.58063
[10]    train-logloss:0.23292   validation-logloss:0.26041
[20]    train-logloss:0.17919   validation-logloss:0.21499
[30]    train-logloss:0.16244   validation-logloss:0.20297
[40]    train-logloss:0.16057   validation-logloss:0.20140
[50]    train-logloss:0.16057   validation-logloss:0.20149
[60]    train-logloss:0.15975   validation-logloss:0.20081
[70]    train-logloss:0.15975   validation-logloss:0.20080
[80]    train-logloss:0.15796   validation-logloss:0.20021
[90]    train-logloss:0.15642   validation-logloss:0.19738
[99]    train-logloss:0.15643   validation-logloss:0.19751

Training complete!
```

> This trains **right inside your notebook** using the open-source XGBoost library. No SageMaker training job, no separate cloud machine, no quotas needed. For the ML Challenge dataset, this is all you need.

Step 8: Get Predictions

```
# Predict on test data
dtest = xgb.DMatrix(test_data_no_target)
predictions = model.predict(dtest)

# Show sample predictions
print("PREDICTIONS (probability of churn):")
print("-" * 55)
for i in range(10):
    pred = predictions[i]
    actual = test_target.iloc[i]
    predicted = "CHURN" if pred > 0.5 else "STAY"
    actual_lbl = "CHURN" if actual == 1.0 else "STAY"
    status = "CORRECT" if predicted == actual_lbl else "WRONG"
    print(f"  Customer {i+1}: {pred:.3f} -> {predicted:5s} (Actual: {actual_lbl:5s}) {s
```

```
[14]: # Predict on test data
dtest = xgb.DMatrix(test_data_no_target)
predictions = model.predict(dtest)

# Show sample predictions
print("PREDICTIONS (probability of churn):")
print("-" * 55)
for i in range(10):
    pred = predictions[i]
    actual = test_target.iloc[i]
    predicted = "CHURN" if pred > 0.5 else "STAY"
    actual_lbl = "CHURN" if actual == 1.0 else "STAY"
    status = "CORRECT" if predicted == actual_lbl else "WRONG"
    print(f"  Customer {i+1}: {pred:.3f} -> {predicted:5s} (Actual: {actual_lbl:5s}) {status}")
```

```
PREDICTIONS (probability of churn):
-------------------------------------------------------
  Customer 1: 0.555 -> CHURN (Actual: STAY ) WRONG
  Customer 2: 0.200 -> STAY  (Actual: STAY ) CORRECT
  Customer 3: 0.776 -> CHURN (Actual: CHURN) CORRECT
  Customer 4: 0.635 -> CHURN (Actual: CHURN) CORRECT
  Customer 5: 0.912 -> CHURN (Actual: CHURN) CORRECT
  Customer 6: 0.932 -> CHURN (Actual: CHURN) CORRECT
  Customer 7: 0.431 -> STAY  (Actual: CHURN) WRONG
  Customer 8: 0.063 -> STAY  (Actual: STAY ) CORRECT
  Customer 9: 0.064 -> STAY  (Actual: STAY ) CORRECT
  Customer 10: 0.007 -> STAY  (Actual: STAY ) CORRECT
```

> Each prediction is a probability between 0 and 1. Above 0.5 = CHURN, below 0.5 = STAY. The model's confidence matters - 0.92 means very confident, 0.51 means barely sure.

Step 9: Evaluate the Model

```
1  from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
2
3  binary_preds = (predictions > 0.5).astype(int)
4
5  print("FULL TEST SET RESULTS:")
6  print(f"  Accuracy:  {accuracy_score(test_target, binary_preds):.1%}")
7  print(f"  Precision: {precision_score(test_target, binary_preds):.1%}")
8  print(f"  Recall:    {recall_score(test_target, binary_preds):.1%}")
9  print(f"  F1 Score:  {f1_score(test_target, binary_preds):.1%}")
```

```
[15]: from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

binary_preds = (predictions > 0.5).astype(int)

print("FULL TEST SET RESULTS:")
print(f"  Accuracy:  {accuracy_score(test_target, binary_preds):.1%}")
print(f"  Precision: {precision_score(test_target, binary_preds):.1%}")
print(f"  Recall:    {recall_score(test_target, binary_preds):.1%}")
print(f"  F1 Score:  {f1_score(test_target, binary_preds):.1%}")
```

```
FULL TEST SET RESULTS:
  Accuracy:  91.6%
  Precision: 92.3%
  Recall:    90.9%
  F1 Score:  91.6%
```

The Results

~92% accuracy. Zero hyperparameter tuning. Trained in seconds. That's the power of XGBoost on tabular data.

Step 10: Save Your Model (Optional)

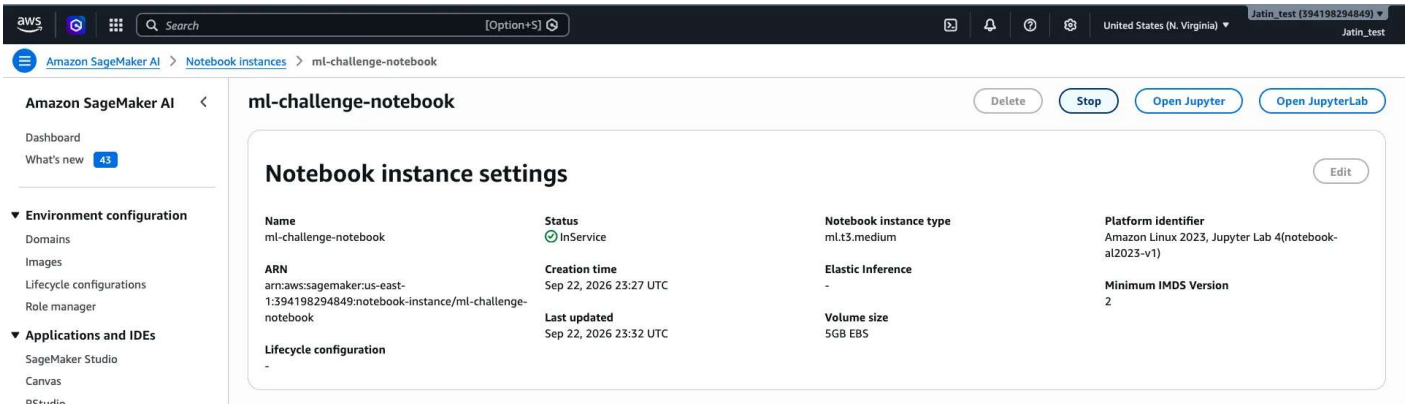
```python
# Save the trained model locally
model.save_model("xgboost_churn_model.json")
print("Model saved! You can reload it anytime with:")
print("  loaded_model = xgb.Booster()")
print("  loaded_model.load_model('xgboost_churn_model.json')")
```



> For the ML Challenge, save your model and predictions. You'll submit a zip file with your code + approach document.

Cleanup

> When you're done for the day, go to the SageMaker console -> Notebook instances -> select your notebook -> **Stop**. This stops the billing. You can restart it anytime later - your files will still be there.Unlike endpoints, notebook instances don't charge when stopped. Just remember to stop, not delete (deleting removes your files). After the challenge you can delete your notebook instance



Your Action Plan

Here's what you should do **right now**:

Resources

- 📺 **Live Session (Sep 21, 5-6 PM IST):** Watch the Recording on Twitch
- **AWS Builder Center:** builder.aws.com

- **Student Rewards Signup:** aws.amazon.com/builder
- **ML Challenge on Unstop:** Registration Page
- **SageMaker Free Tier:** aws.amazon.com/sagemaker/pricing
- **SageMaker SDK v3 Tutorial:** sagemaker.readthedocs.io

🤝 Let's Connect

If you found this helpful, connect with me on LinkedIn! or instagram I regularly post about AWS, ML, and developer tools.

Jatin Mehrotra - Developer Advocate, AWS

`# builder-center` `# aws-community` `# getting-started` `# aws-free-tier` `# machine-learning`

Any opinions in this article are those of the individual author and may not reflect the opinions of AWS.

98 126

Enjoyed reading this content? Let the author know!

Your likes, comments, shares, and saves help creators reach more builders.

Explore more articles

AWS Student Rewards: Your Step-by-Step Guide to All 21 Badges (+ Tips & Tricks)



Adidev V.N

29 19

Introducing Polls on AWS Builder Center



Rick Suttles

AWS Employee

2320 1657

AWS en Nerdearla: Aprende, construye y gana 🚀



Ana Cunha

AWS Employee

670 95

Related topics

#aws-community #aws-free-tier #builder-center

Comments (126)

Sign in to comment

Sign in

Sort by Newest



Lokesh Akurathi • 7h ago

aws freetier is not providing acess to large instances like m5.2xlarge with 32 gb vram. how can we train model

0 0 Reply



Divya Asnani • 15h ago

Very helpful, thank you!

0 0 Reply



Ritesh Bhende • 19h ago

1. Article 1 — Spring Boot to AWS
2. Article 2 — My First AWS Project
3. Article 3 — Spring Boot on AWS
4. Article 4 — Serverless REST API
5. Article 5 — My AWS Learning Journey

Please Like 👍 all 5 if you find them useful. Your support means a lot!

0 0 Reply



Jahnavi Reddy P • 21h ago

Student Builder Group Leader

Helpful guide, great demo!

1 0 Reply



Deepti • 1d ago

Helpfull thank you !!

1 0 Reply



Adi • 1d ago

Thank you.

1 0 Reply



Balavignesh • 1d ago

A whole bunch of thanks

1 0 Reply



Vishwanath mandala • Modified 1d ago

Informative Content

0 0 Reply



Andrés • 1d ago